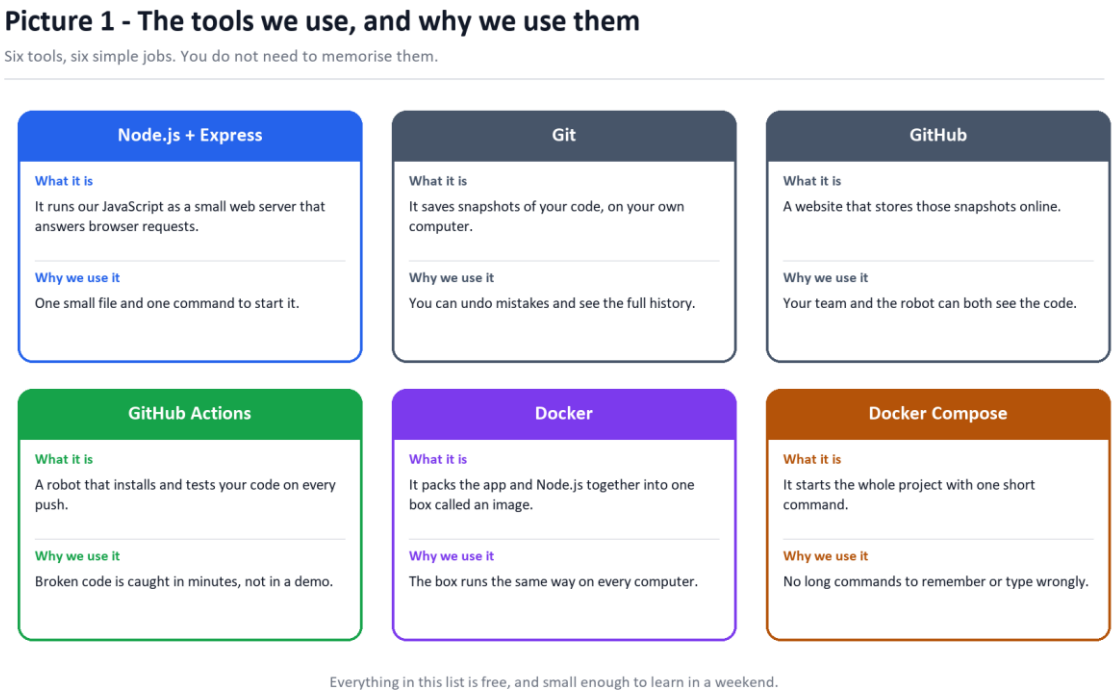
TaskBoard Explained for Beginners

A short guide: what this project uses, where the files live, and how everything connects.

TaskBoard is a tiny task list website. You add a task, click it to mark it done, and delete it when you are finished. The app is small on purpose: the real subject of this project is the DevOps around it - Git, GitHub Actions, Docker and Docker Compose.

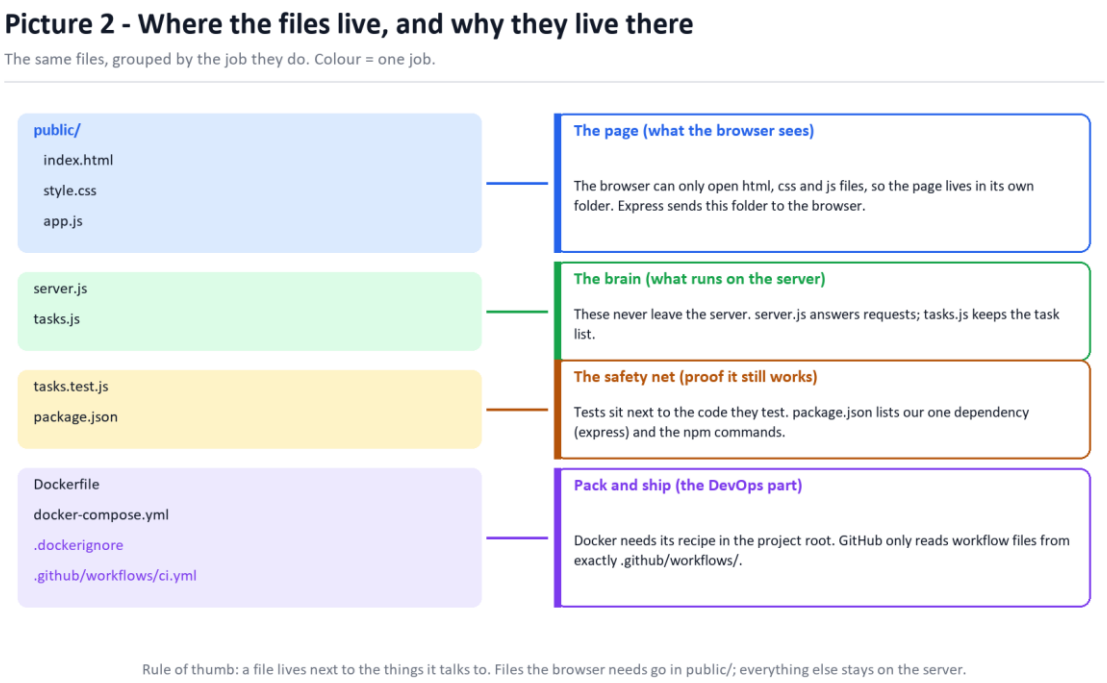
1. What we use, and why



Picture 1 - The six tools, what each one is, and why we chose it.

Tool	What it is, and why we use it
Node.js + Express	Runs our JavaScript as a small web server. One file, one command.
Git	Saves snapshots of your code on your computer, so you can undo mistakes.
GitHub	Stores those snapshots online, so your team and the robot can see them.
GitHub Actions	A robot that installs and tests the code on every push.
Docker	Packs the app and Node.js into one image, so it runs the same everywhere.
Docker Compose	Starts the whole project with one short command.

2. Where the files live, and why



Picture 2 - The same files, grouped by the job they do.

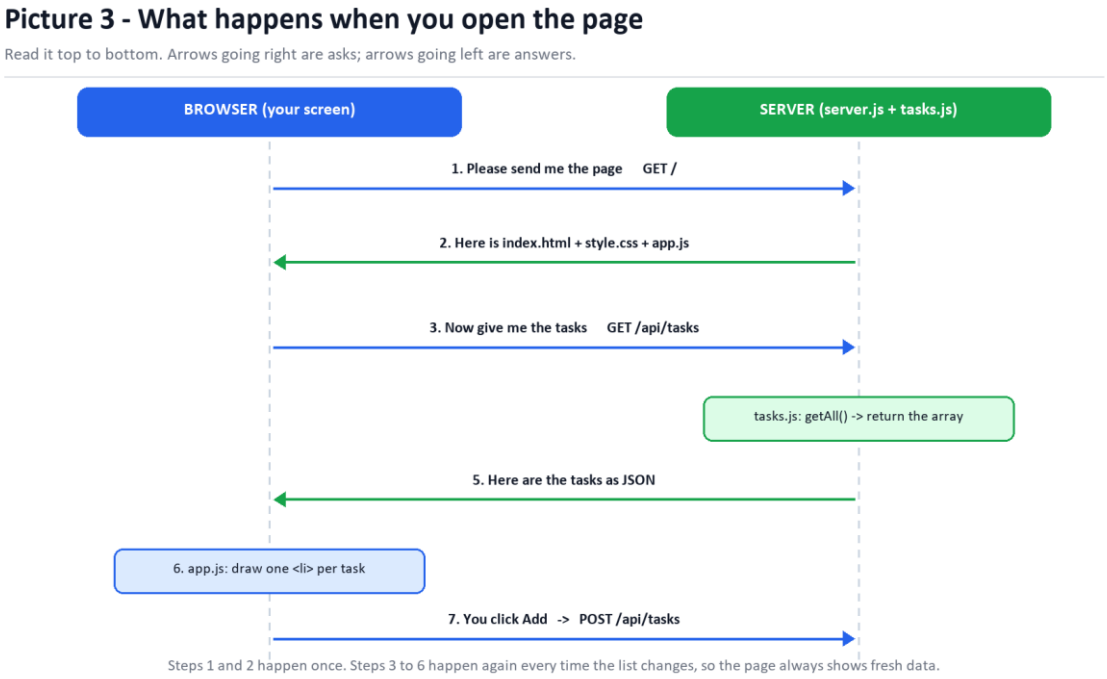
Where	What lives there	Why there
public/	index.html, style.css, app.js	The browser needs these files.
Root files	server.js, tasks.js	The server and the data it keeps.
Root files	tasks.test.js, package.json	The tests and the dependency list.
Root files	Dockerfile, compose files	Docker needs these names here.
.github/workflows/	ci.yml	GitHub reads workflows only from here.
docs/	this guide and pictures	Keeps the explanation with the code.

Rule of thumb: if the browser needs a file it goes in public/. Everything else stays on the server, and the tools demand the rest of the names.

3. How the app works

There are three layers, and it helps to keep them apart: the browser asks, the server decides, and tasks.js remembers.

Layer	Runs in	Its job
The page	Your browser	Shows things and reacts to your clicks.
The server	Node.js	Answers the asks and makes the decisions.
The data	Node.js too	Remembers the list of tasks.

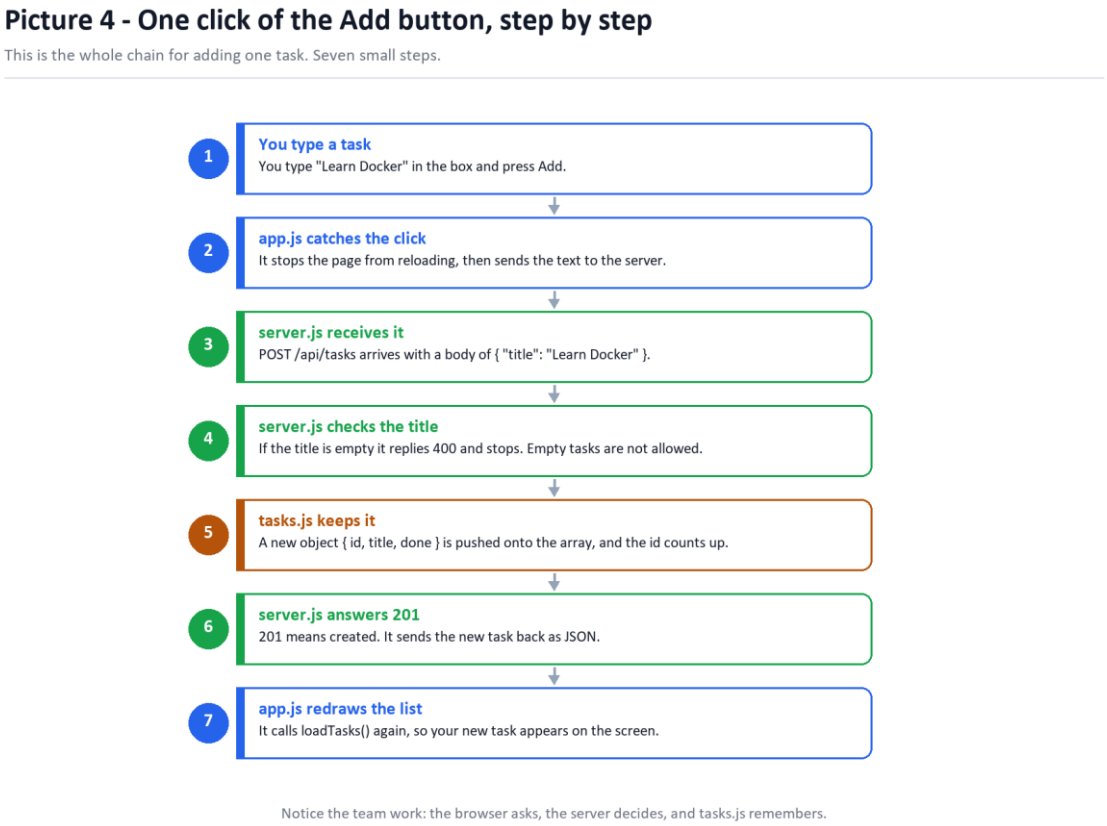


Picture 3 - Opening the page: the browser asks, the server answers.

Steps 1 and 2 happen once. Steps 3 to 6 repeat every time the list changes, so the screen always shows fresh data.

4. One click of the Add button

This is the whole chain for adding one task. It touches all three layers.



Picture 4 - Seven small steps, from your click to the task on screen.

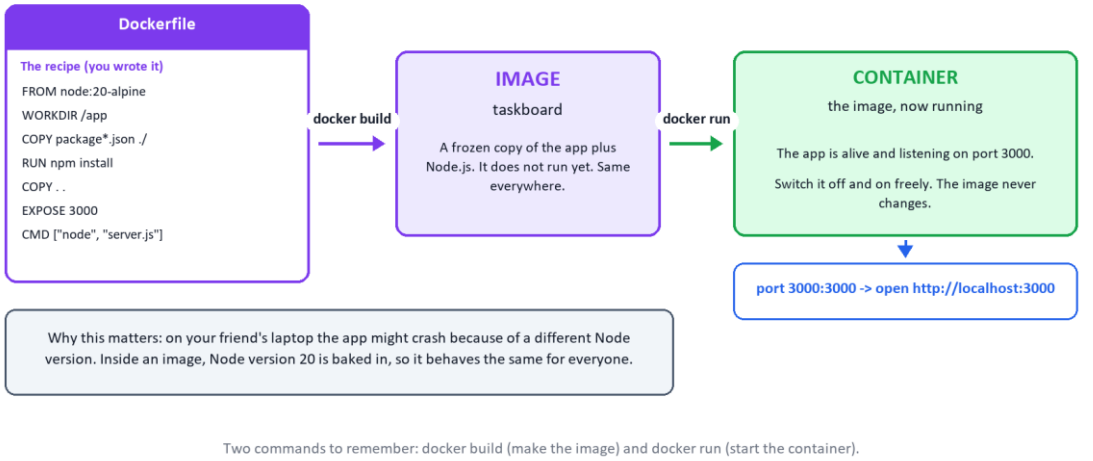
Ask	Address	What it does
GET	/api/tasks	Give me all the tasks.
POST	/api/tasks	Create a task.
PATCH	/api/tasks/3	Mark task 3 done, or not done again.
DELETE	/api/tasks/3	Delete task 3.
GET	/health	Are you alive? Used by Docker.

5. Docker: recipe, image, container

Docker packs the app together with the exact Node.js version it needs, so it behaves the same on every computer.

Picture 5 - Docker: a recipe, a frozen box, and a running app

Like cooking: the recipe, the frozen meal, and the meal served on a plate.



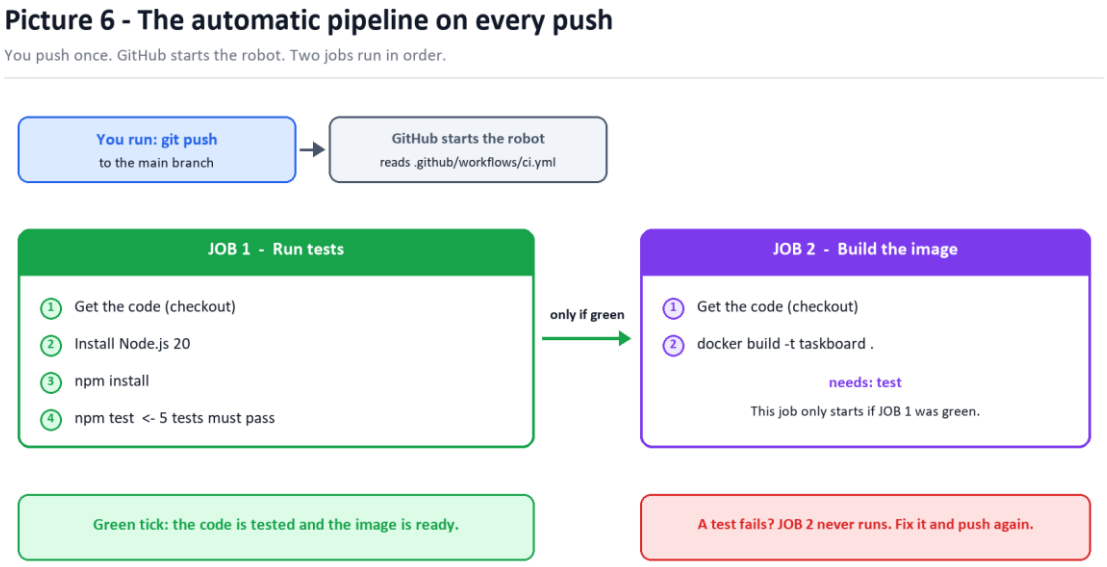
Two commands to remember: docker build (make the image) and docker run (start the container).

Picture 5 - The recipe, the frozen image, and the running container.

Command	What it does
docker build -t taskboard .	Makes the image from the Dockerfile.
docker run -p 3000:3000 taskboard	Starts a container from the image.
docker compose up --build	Builds and starts everything in one command.

6. The automatic pipeline

Every time you push to main, GitHub runs the workflow file on a fresh computer in the cloud. Nobody clicks anything.



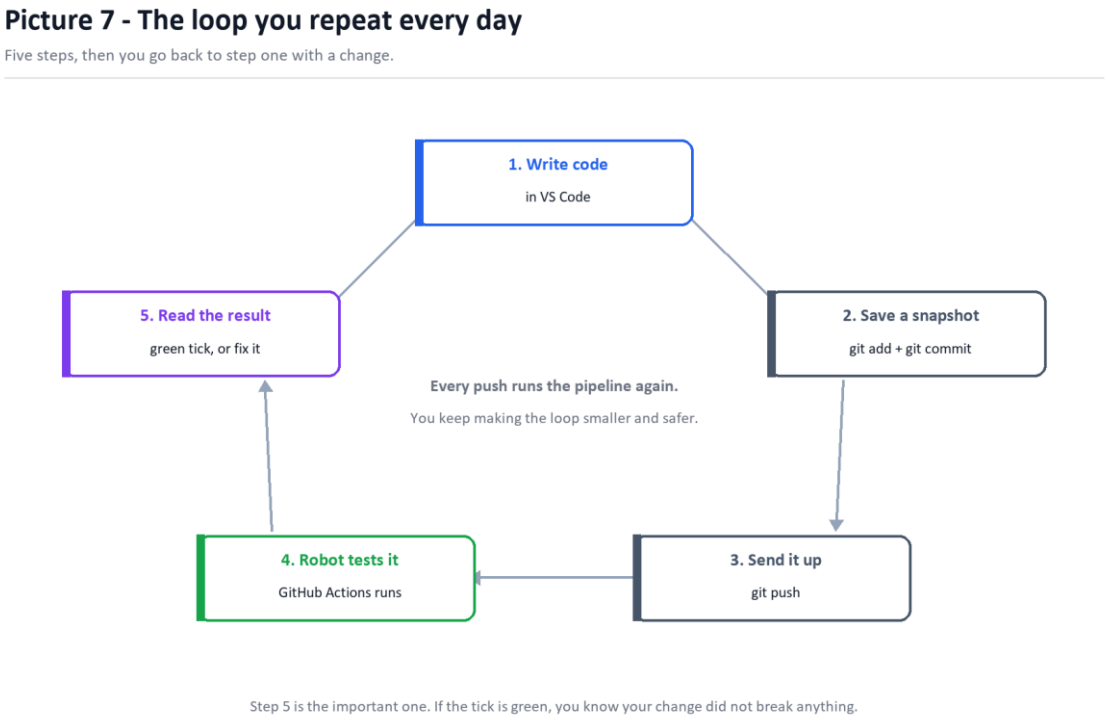
Picture 6 - The pipeline, including what happens when a test fails.

Job	Steps inside it	Why it exists
JOB 1 test	checkout, Node 20, npm install, npm test	Proves the change did not break the tests.
JOB 2 docker	checkout, docker build -t taskboard .	Turns tested code into a ready-to-ship image.

The line that matters is needs: test. If job 1 fails, job 2 never runs.

7. The loop you repeat every day

This is the everyday rhythm of working on the project.



Picture 7 - Write, save, push, wait, read the result, repeat.

- Change something in VS Code.
- Save a snapshot: git add . then git commit -m "my change".
- Send it up: git push - and the pipeline starts by itself.
- Open the Actions tab. A green tick means your change is safe.
- A red cross means a test failed: read it, fix it, and start again.

8. Cheat sheet

The three chains

Chain	In one line
Docker	Dockerfile -> build -> image -> run -> container -> port 3000 -> browser
Pipeline	git push -> job 1 tests -> job 2 builds the image
Daily loop	write -> commit -> push -> wait for the green tick -> repeat

Words you need

Word	In simple words
Server	A program that waits for asks and answers them.
Browser	The program on your screen that makes the asks.
API	The fixed list of asks the server understands.
JSON	A simple text way of writing data.
Port	A numbered door on a computer. Ours is 3000.
Commit	One saved snapshot of your code, with a message.
Push	Sending your saved snapshots up to GitHub.
Image / Container	The frozen package, and that package running.
CI / CD	Automatic testing, then automatic packaging.

Commands you will use

Command	What it does
npm install	Downloads the dependency. Once per computer.
npm start	Runs the app on http://localhost:3000.
npm test	Runs the five tests.
docker compose up --build	Runs the project inside a container.
docker compose down	Stops and removes the container.
git add . then git commit	Saves a snapshot of your work.